Actividad 3. Taller resuelto
Reconocimiento de dígitos manuscritos con redes profundas

Informe de ejecución

Asignatura: Inteligencia Computacional Aplicada
Docente: Cesar Andrey Perdomo Charry
Estudiante: Sergio Santiago Duarte Rojas
Código: 20261595002

Fecha: 10 de octubre de 2026

Universidad Distrital Francisco José de Caldas

Índice

1. Introducción **2**

2. Marco teórico (resumen) **2**
2.1. De vectores a imágenes 2
2.2. Pérdida y optimizador 2

3. Metodología **3**
3.1. Datos y particiones 3
3.2. Arquitecturas 3
3.3. Entrenamiento y evaluación 3

4. Resultados **4**
4.1. Curvas de aprendizaje 4
4.2. Evaluación en prueba 4
4.3. Tasa de aprendizaje 5
4.4. Modelo guardado 6

5. Análisis **7**
5.1. Por qué gana la CNN con menos parámetros 7
5.2. Qué dicen las curvas 7
5.3. Qué dígitos se confunden 7
5.4. Qué muestra la tasa de aprendizaje 7

6. Conclusiones **8**

7. Referencias **8**

1. Introducción

El Taller 2 cerró con un perceptrón multicapa entrenado por retropropagación sobre vectores de pocas entradas. Este taller lleva la misma idea a imágenes. El cuaderno del docente entrena dos redes sobre MNIST (LeCun et al., 1998), 70 000 dígitos manuscritos de 28×28 píxeles, primero un perceptrón multicapa que trata la imagen como un vector de 784 números y después una red convolucional que la trata como una imagen. El informe documenta la ejecución, confirma los puntos de control y explica lo que muestran las curvas, la matriz de confusión y el experimento con la tasa de aprendizaje.

El cuaderno se ejecutó completo y sin modificaciones, con PyTorch 2.12 (Paszke et al., 2019) sobre una GPU NVIDIA y semilla 42. MNIST se descargó con torchvision, sin recurrir al conjunto de respaldo que el cuaderno trae para cuando no hay conexión. Las diez verificaciones de los cinco puntos de control dieron `[OK]`.

2. Marco teórico (resumen)

2.1 De vectores a imágenes

El MLP del Taller 2 recibe un vector y lo pasa por capas densas, en las que cada neurona se conecta con todas las entradas. Aplicado a una imagen, eso obliga a aplanarla, con lo que se pierde la noción de vecindad entre píxeles y el número de pesos crece con el tamaño de la imagen. Una capa convolucional desliza un filtro pequeño, aquí de 3×3, por toda la imagen y calcula en cada posición una suma ponderada con los mismos pesos. Un filtro aprende entonces un rasgo local, un borde o un trazo, y lo detecta en cualquier posición. Una capa con c_{in} canales de entrada, c_{out} filtros y tamaño k tiene $c_{out}(c_{in}k^2 + 1)$ parámetros, sin importar el tamaño de la imagen. Sin relleno (píxeles de valor cero añadidos alrededor de la imagen) la salida pierde $k - 1$ píxeles por eje. La activación ReLU, $\max(0, z)$, es la misma del Taller 2. El *max pooling* toma el máximo de cada ventana de 2×2 y reduce la imagen a la mitad por lado, lo que da cierta tolerancia a desplazamientos de un píxel y abarata las capas siguientes.

2.2 Pérdida y optimizador

Con diez clases la salida de la red son diez *logits* z_k. La función softmax los convierte en probabilidades, $p_k = e^{z_k} / \sum_j e^{z_j}$, y la pérdida es la entropía cruzada $L = -\ln p_y$, donde y es la clase correcta. Es la misma cadena de retropropagación del Taller 2 con otra activación de salida y otra medida de error. Para una red que reparte la probabilidad por igual entre las diez clases vale $\ln 10 = 2.303$, una referencia útil para leer las tablas. Adam (Kingma y Ba, 2015) combina el momento del Taller 2, un promedio móvil del gradiente, con un segundo promedio móvil del gradiente al cuadrado que normaliza el paso de cada peso, de modo que una misma tasa de aprendizaje sirve para pesos con gradientes de magnitudes muy distintas. El *dropout* (Srivastava et al., 2014) apaga al azar una fracción p de las activaciones en cada paso de entrenamiento y reescala las restantes por $1/(1-p)$, para que la evaluación, que usa todas, vea activaciones de la misma escala.

3. Metodología

3.1 Datos y particiones

MNIST trae 60 000 imágenes de entrenamiento y 10 000 de prueba. El cuaderno aparta 5 000 imágenes del entrenamiento como validación, deja 55 000 para entrenar y reserva la prueba para la evaluación final. Los píxeles se llevan de 0 a 255 al rango 0 a 1 y después se normalizan con la media (0.1307) y la desviación (0.3081) calculadas solo sobre el entrenamiento. El desbalance entre clases es moderado, la más numerosa tiene un 25 % más imágenes que la menos numerosa. La Figura 1 muestra las primeras dieciséis.

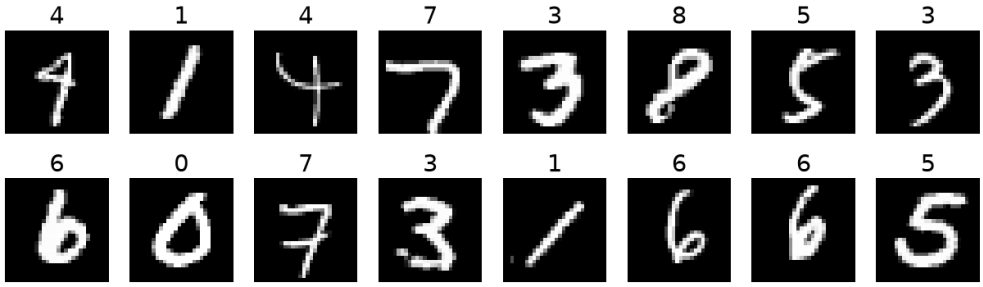


Figura 1: Primeras dieciséis imágenes de entrenamiento con su etiqueta

3.2 Arquitecturas

La Tabla 1 y la Figura 2 describen las dos redes. El MLP aplana la imagen y tiene una capa oculta de 128 neuronas con ReLU, 101 770 parámetros. La CNN tiene dos bloques de convolución 3×3 seguida de ReLU y *max pooling*, un *dropout* de 0.25 y una capa lineal, 12 810 parámetros, 7.9 veces menos que el MLP. Ambas devuelven diez *logits* y la softmax va dentro de la pérdida.

Tabla 1: Capas de las dos redes. En la CNN las convoluciones no usan relleno, así que cada una quita un píxel por lado

Red	Capa	Parámetros
MLP	Aplanar, 784	0
	Lineal 784 a 128 con ReLU	100 480
	Lineal 128 a 10	1 290
CNN	Convolución 3×3, 16 filtros, ReLU, salida $16 \times 26 \times 26$	160
	Max pooling 2×2, salida $16 \times 13 \times 13$	0
	Convolución 3×3, 32 filtros, ReLU, salida $32 \times 11 \times 11$	4 640
	Max pooling 2×2, salida $32 \times 5 \times 5$	0
	Aplanar, 800, y dropout 0.25	0
	Lineal 800 a 10	8 010

3.3 Entrenamiento y evaluación

Las dos redes se entrenan 3 épocas con Adam, tasa 10^{-3}, lotes de 64 imágenes en orden aleatorio y entropía cruzada. Conviene tener presente cómo mide el cuaderno cada cifra. La pérdida y la exactitud de entrenamiento son el promedio de los lotes de la época, calculado mientras los pesos todavía cambian y con el *dropout* activo. La pérdida y la exactitud de validación se miden al final de

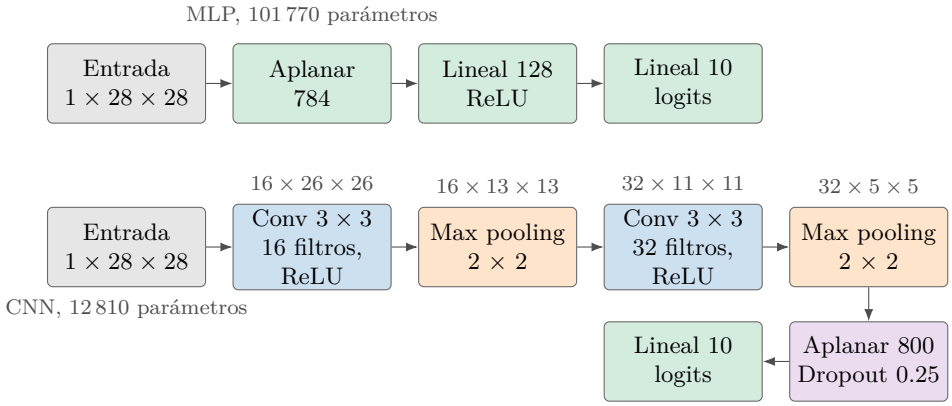


Figura 2: Las dos redes del cuaderno. Encima de cada bloque de la CNN va el tamaño de su salida, canales por alto por ancho

la época, con la red ya actualizada y en modo de evaluación. Las dos columnas no son comparables al detalle y eso explica parte de lo que se ve en las curvas. La prueba se evalúa una vez con la CNN entrenada y otra vez solo para comprobar el modelo recargado desde disco.

El cuaderno pide anotar una predicción antes del experimento con la tasa de aprendizaje, que repite el entrenamiento de la CNN durante 2 épocas con 10^{-1}, 10^{-3} y 10^{-5}. Lo esperado era que 10^{-1} fuera demasiado grande para Adam y la red no aprendiera, que 10^{-5} aprendiera pero quedara lejos en 2 épocas y que 10^{-3} repitiera el resultado principal.

4. Resultados

4.1 Curvas de aprendizaje

La Tabla 2 y la Figura 3 recogen las tres épocas de cada red. La CNN termina con 98.64 % de validación frente a 97.02 del MLP, 1.6 puntos más, y su pérdida de validación es la mitad. Las dos cifras quedan dentro de los rangos orientativos del cuaderno, 96 a 98 % para el MLP y 98.5 a 99.2 para la CNN.

Tabla 2: Pérdida y exactitud por época. La exactitud está en porcentaje. Las columnas de entrenamiento son promedios de la época y las de validación se miden al final de ella

		Entrenamiento		**Validación**	
Red	**Época**	**Pérdida**	**Exactitud**	**Pérdida**	**Exactitud**
MLP	1	0.2652	92.28	0.1604	95.22
	2	0.1174	96.49	0.1148	96.56
	3	0.0821	97.49	0.1023	97.02
CNN	1	0.2305	93.01	0.0864	97.48
	2	0.0813	97.51	0.0645	98.00
	3	0.0620	98.07	0.0482	98.64

4.2 Evaluación en prueba

La CNN acierta 98.65 % de las 10 000 imágenes de prueba, 135 errores, con pérdida 0.0381. La Figura 4 muestra la matriz de confusión y la Tabla 3 el acierto por clase, que va de 97.9 % en el 8

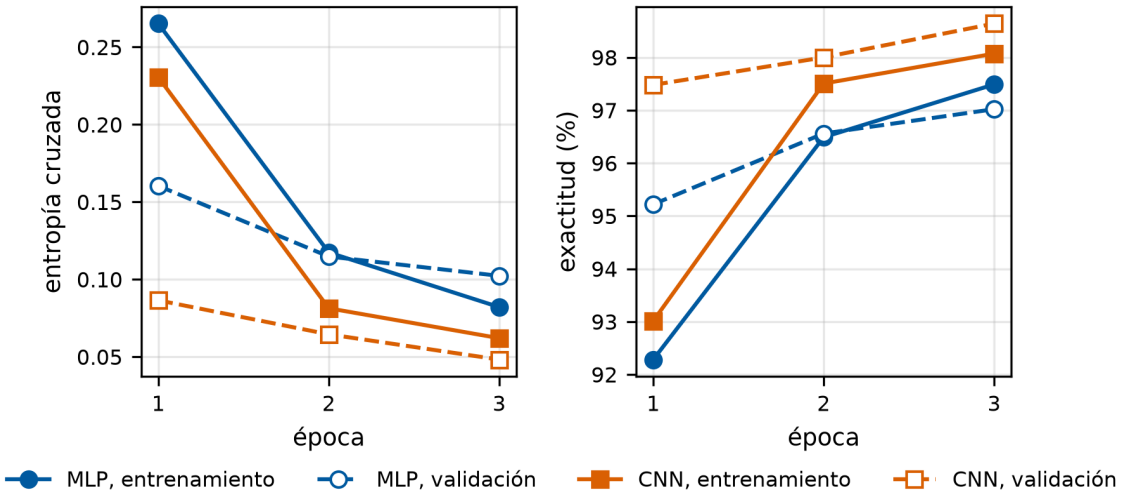


Figura 3: Pérdida y exactitud por época de las dos redes. Línea continua y marcador lleno para el entrenamiento, discontinua y marcador hueco para la validación

a 99.7 en el 1. Los errores se reparten por 54 de las 90 celdas fuera de la diagonal. Los tres pares más frecuentes, 4 tomado por 9, 2 por 7 y 7 por 2, reúnen 29 errores, el 21 % del total. La columna del 7 recibe 32 errores de otras clases y por eso ese dígito tiene la precisión más baja. La Figura 5 muestra los diez primeros errores.

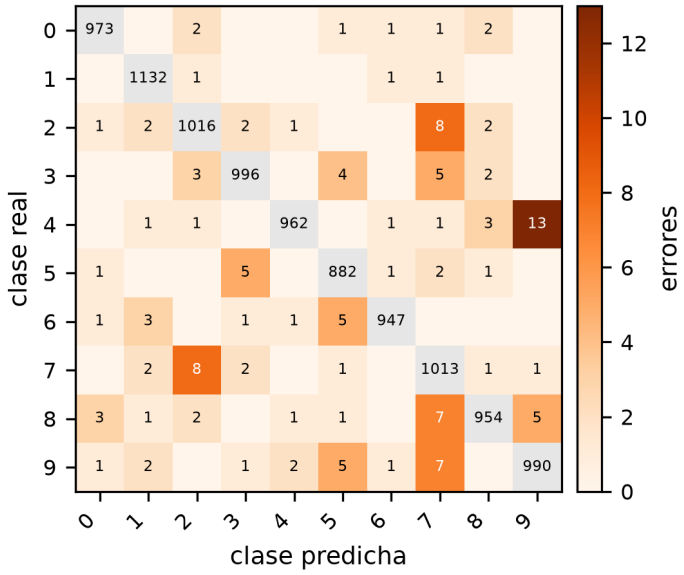


Figura 4: Matriz de confusión de la CNN sobre las 10 000 imágenes de prueba. El color codifica solo los errores y la diagonal va en gris para que los aciertos no aplanen la escala

4.3 Tasa de aprendizaje

La Tabla 4 resume el experimento de 2 épocas y confirma la predicción. Con 10^{-1} la red queda en 10.42 % y su pérdida de validación se queda a 2 milésimas de $\ln 10$. Con 10^{-5} llega a 81.22 % con una pérdida todavía alta y con 10^{-3} alcanza 98.10 %.

Tabla 3: Acierto por clase en prueba. El acierto es la fracción de imágenes de la clase bien clasificadas y la precisión la fracción de predicciones de esa clase que eran correctas

Dígito	Imágenes	Aciertos	Acierto (%)	Precisión (%)
0	980	973	99.3	99.3
1	1135	1132	99.7	99.0
2	1032	1016	98.4	98.4
3	1010	996	98.6	98.9
4	982	962	98.0	99.5
5	892	882	98.9	98.1
6	958	947	98.9	99.5
7	1028	1013	98.5	96.9
8	974	954	97.9	98.9
9	1009	990	98.1	98.1

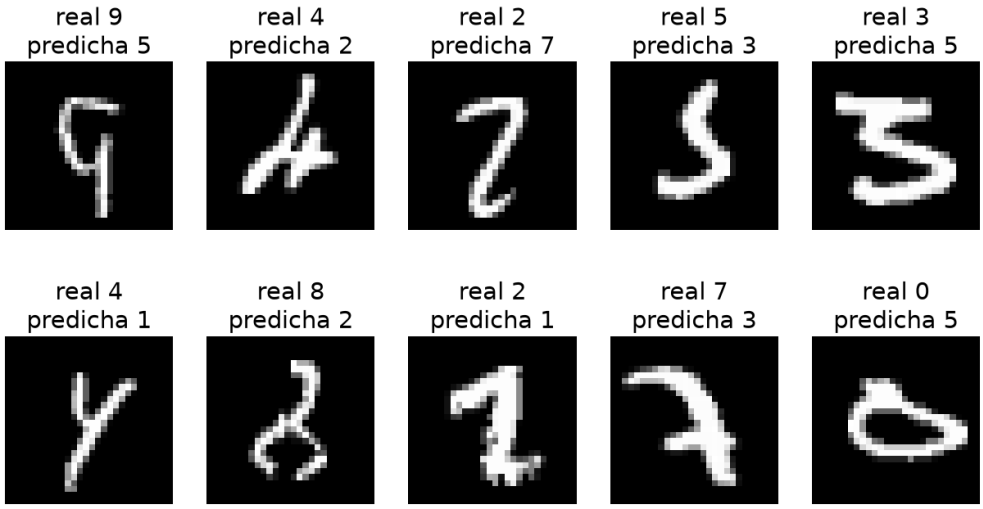


Figura 5: Los diez primeros errores de la CNN en prueba, con la clase real y la predicha

Tabla 4: CNN tras 2 épocas con tres tasas de aprendizaje. La pérdida de entrenamiento es el promedio de cada época

Tasa	Pérdida ent. época 1	Pérdida ent. época 2	Exactitud val. (%)	Pérdida val.
1×10^{-1}	2.41	2.31	10.42	2.3045
1×10^{-3}	0.23	0.08	98.10	0.0580
1×10^{-5}	2.16	1.55	81.22	1.1829

4.4 Modelo guardado

Los pesos de la CNN se guardaron en disco, se cargaron en una red nueva y la exactitud de prueba fue la misma, así que el modelo se puede reutilizar sin entrenarlo otra vez.

5. Análisis

5.1 Por qué gana la CNN con menos parámetros

El MLP gasta 100 480 de sus parámetros en conectar cada píxel con cada neurona oculta, y cada uno de esos pesos solo sirve para una posición de la imagen. La primera capa de la CNN tiene 160 parámetros porque sus 16 filtros se reutilizan en las 676 posiciones de la imagen, y el *pooling* hace que un trazo movido un píxel active casi lo mismo. Con 7.9 veces menos parámetros la CNN mejora la validación en 1.6 puntos y reduce la pérdida a la mitad. Esa es la explicación habitual, el cuaderno no incluye un experimento que la aísle, pero la diferencia de resultados es consistente con ella.

5.2 Qué dicen las curvas

En las dos redes la validación supera al entrenamiento en la primera época, 95.22 frente a 92.28 en el MLP y 97.48 frente a 93.01 en la CNN. El MLP no tiene *dropout*, así que la causa principal no puede ser esa. Es la forma de medir descrita en la Sección 3.3. La exactitud de entrenamiento es un promedio de toda la época, que arranca con una red casi al azar, mientras que la validación se mide con la red que queda al final. A medida que la red cambia menos de una época a otra la diferencia se cierra, y en la tercera época el MLP ya supera a la validación por 0.47 puntos. En la CNN la validación sigue por encima en las tres épocas porque el *dropout* añade una segunda causa, entrena con el 75 % de las 800 características y evalúa con todas.

El MLP no muestra sobreajuste en tres épocas. Su pérdida de validación sigue bajando y la diferencia de 0.47 puntos en la última época es una brecha de generalización pequeña, no una divergencia entre las curvas como la que el Taller 2 mostró en Breast Cancer. Con más épocas sería el primer sitio donde mirar.

5.3 Qué dígitos se confunden

Los errores no se concentran en unos pocos pares. Ocupan 54 de las 90 celdas fuera de la diagonal y los tres pares más frecuentes solo reúnen el 21 % de ellos. Lo que sí se ve es una estructura. Los pares 4 con 9 y 2 con 7 son dígitos que difieren en un trazo, un 4 con la parte superior cerrada se convierte en un 9 y un 7 con el trazo horizontal curvo se parece a un 2. La columna del 7 recibe más errores que ninguna otra, 32, lo que sugiere que la forma del 7, un trazo horizontal y otro inclinado, aparece como fragmento en dígitos de otras clases mal escritos. De los diez errores de la Figura 5 solo uno pertenece a los tres pares principales y varios son dígitos que una persona también dudaría en clasificar.

5.4 Qué muestra la tasa de aprendizaje

Con 10^{-1} la pérdida de entrenamiento de la primera época, 2.41, supera a $\ln 10$. Adam da pasos de tamaño cercano a la tasa sin importar la magnitud del gradiente, así que los pesos saltan a valores que ya no dependen de la imagen y la red colapsa a una salida casi constante. Una exactitud de 10.42 % corresponde a predecir casi siempre la misma clase, y una pérdida a 2 milésimas de $\ln 10$ dice que la salida constante reparte la probabilidad casi por igual entre las diez clases. Con 10^{-5} la pérdida sí baja época a época, así que la red aprende, pero necesitaría muchas más épocas. El valor intermedio de 10^{-3} repite la lección del Taller 2 con la tasa del MLP, demasiado grande no aprende y demasiado pequeña no llega a tiempo.

La corrida con 10^{-3} de este experimento no repite las cifras de la Tabla 2 en sus dos primeras épocas. La función que fija la semilla reinicia los generadores de PyTorch pero no el generador propio que el cuaderno le entrega al cargador de datos, así que el orden de los lotes cambia de una corrida a otra. Además algunas operaciones de la convolución en GPU no son deterministas, y por eso la CNN cambia en la segunda cifra decimal entre ejecuciones completas del cuaderno mientras que el MLP repite sus cifras.

6. Conclusiones

La red convolucional del cuaderno alcanza 98.65 % en prueba con 12 810 parámetros y supera al MLP de 101 770 parámetros en 1.6 puntos de validación, lo que se explica porque reutiliza cada filtro en toda la imagen y el *pooling* tolera desplazamientos de un píxel.

La validación por encima del entrenamiento en la primera época se debe a que el cuaderno promedia la exactitud de entrenamiento durante la época y mide la validación al final. El *dropout* alarga ese efecto en la CNN. El MLP no muestra sobreajuste en tres épocas, su pérdida de validación sigue bajando.

Los 135 errores de prueba se reparten por 54 celdas de la matriz. Los pares más frecuentes, 4 con 9 y 2 con 7, son dígitos que difieren en un trazo, y el acierto por clase va de 97.9 a 99.7 %.

La tasa de aprendizaje decide entre no aprender, aprender despacio y aprender bien. Con Adam el valor de 10^{-3} es el que funciona y 10^{-1} colapsa la red a una salida constante.

Las diez verificaciones se cumplieron y el modelo guardado reproduce la exactitud de prueba al recargarse. La semilla fija la inicialización pero no el orden de los lotes, y la GPU añade pequeñas diferencias, así que las cifras de la CNN varían en la segunda decimal entre corridas.

7. Referencias

- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
- Paszke, A. et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.